MCU TINKERING LAB

robocar-unified

Single-Board AI Robot Car

Assembly, Wiring & Firmware Build Guide

A hands-on guide to assembling the consolidated robocar on a **Seeed Studio XIAO ESP32-S3 Sense**. Camera capture, Gemini AI planning, motor control, and peripherals all run on one module.

Target MCU	ESP32-S3 (8 MB PSRAM / flash)
Toolchain	ESP-IDF v5.4 (containerized)
Difficulty	Intermediate · ~2–3 h

Pin data auto-generated from `main/pin_config.h` via `just robocar-unified::gen-pin-defs`.
All components must share a common ground.

Contents

1 · Overview

The **robocar-unified** project consolidates the original dual-ESP32 robocar (a Heltec main controller plus an ESP32-CAM) onto a single **XIAO ESP32-S3 Sense**. One module now handles camera capture, AI inference, motor control, and all peripherals, using dual-core affinity to keep motor timing isolated from bursty network and vision work.

The control system is **hierarchical**: a slow **planner** (Core 1) calls Google's Gemini Robotics-ER to emit structured goals, and a fast **reactive executor** (~30 Hz, Core 0) drives the robot smoothly toward those goals while an ultrasonic sensor provides an independent obstacle reflex. The planner is not on a fixed schedule: the board boots **dormant** and asks Gemini only when something happened, backing off 15 s ⟶ 300 s when nothing does.

Core 0 — real-time
Reactive executor (visual servo, heading hold, motor PWM), peripheral I/O, command console, and the ultrasonic obstacle reflex.

Core 1 — bursty I/O
Planner (Gemini calls), OV3660 camera DMA, audio, WiFi / MQTT / OTA.

What you get
A two-wheel-drive car that captures frames, asks Gemini what to do, and drives toward goals — with pan/tilt camera, status LEDs, an OLED display, buzzer feedback, WiFi provisioning over the USB console, and over-the-air updates.

2 · Bill of Materials

Qty	Component	Notes
1	XIAO ESP32-S3 Sense	MCU + OV3660 camera + PDM mic + 8 MB PSRAM, USB-C
1	TCA9548A I²C multiplexer	Breakout, address 0x70
1	PCA9685 16-ch PWM driver	Breakout, address 0x40
1	TB6612FNG dual motor driver	Breakout
1	SSD1306 OLED display	128×64, I²C, address 0x3C
1	MCP23017 GPIO expander	Breakout, address 0x20 — optional
1	MAX98357A I²S class-D amplifier	Mono, for voice output
1	Speaker	4–8 Ω, 2–3 W
1	Ultrasonic rangefinder	**3.3 V variant**: HC-SR04P / RCWL-1601 / US-100
2	RGB LED	Common-anode
2	SG90 micro servo	Pan / tilt
2	DC gear motor + wheel	~3–6 V hobby motors
1	Piezo buzzer	Passive
1	100 Ω resistor	In series with buzzer
1	Electrolytic capacitor	≥470 µF — for MAX98357A supply
2	18650 Li-ion cell + holder	Battery pack
1	XL6009 boost converter	Regulated to 5 V
—	Chassis, wiring, headers	2WD car chassis, jumper wires, standoffs

Tools required

Soldering iron + solder, wire strippers, small screwdriver set, multimeter (for verifying 5 V rail and continuity), a USB-C cable, and a computer with Docker (for the containerized firmware build).

> **Sensor voltage — read this**
> The ultrasonic sensor **must be a 3.3 V-compatible module** (HC-SR04P, not the classic 5 V HC-SR04). The XIAO's GPIOs are not 5 V-tolerant — a 5 V ECHO line can damage the board.

3 · System Architecture

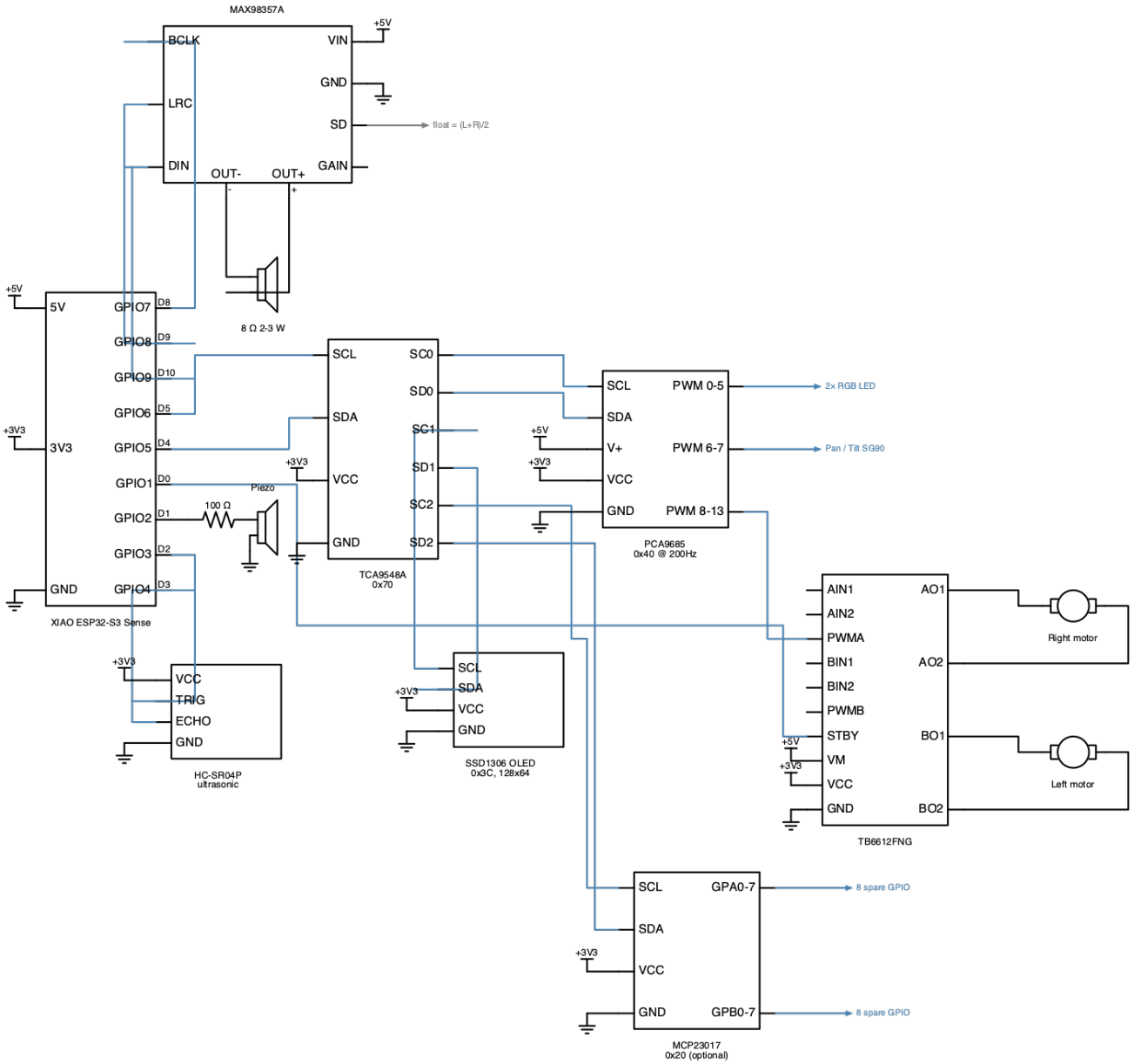

Figure 1: Full wiring schematic. Source: `docs/schematics/circuits/robocar_unified.py`.

Everything on the I²C bus hangs off the **TCA9548A multiplexer** — the ESP32-S3 never talks to the PCA9685 or OLED directly. This lets devices that would otherwise share addresses coexist, and keeps the two 400 kHz devices on separate channels. The ESP32-S3 itself drives only five groups of signals directly: I²C (GPIO5/6), the motor-enable STBY line (GPIO1), the buzzer (GPIO2), the ultrasonic sensor (GPIO3/4), and **I²S audio** (GPIO7/8/9) to the MAX98357A amplifier.

4 · Wiring Reference

4.1 · XIAO ESP32-S3 GPIO assignments

Only 11 GPIOs are exposed on the XIAO headers. Camera pins are internal to the Sense module and do not conflict.

XIAO Pin	GPIO	Function	Notes
D0	GPIO1	TB6612FNG STBY	HIGH = motors enabled
D1	GPIO2	Piezo buzzer	LEDC PWM · 100 Ω in series
D2	GPIO3	Ultrasonic TRIG	10 µs pulse output
D3	GPIO4	Ultrasonic ECHO	Pulse width in (RMT RX)
D4	GPIO5	**I²C SDA**	to TCA9548A
D5	GPIO6	**I²C SCL**	to TCA9548A
D6	GPIO43	USB Serial TX	Debug console
D7	GPIO44	USB Serial RX	Debug console
D8	GPIO7	**I²S BCLK**	to MAX98357A — bit clock
D9	GPIO8	**I²S LRCLK**	to MAX98357A — word select
D10	GPIO9	**I²S DIN**	to MAX98357A — serial data

I²C runs at **400 kHz**.

> **GPIO budget fully allocated**
> There are no spare header pins. Additional digital I/O must go through the MCP23017 on TCA9548A channel 2.

4.2 · I²C topology (TCA9548A @ 0x70)

Select the channel on the multiplexer **before** addressing any downstream device. The MCP23017 on ch2 is optional — the firmware boots fine without it.

Channel	Device	Address
ch0	PCA9685 PWM driver (motors, servos, LEDs)	0x40 @ 200 Hz
ch1	SSD1306 OLED display (128×64)	0x3C
ch2	MCP23017 GPIO expander (optional)	0x20
ch3–7	*reserved — IMU / ToF / future sensors*	—

4.3 · PCA9685 channel map (0x40, 200 Hz)

All motor direction, motor PWM, servo, and LED outputs go through the PCA9685. Motor direction pins use PCA9685 “full-on” (4096) / “full-off” (0); PWM pins use the full 12-bit range (0–4095).

Ch	Signal
0	Left LED — R
1	Left LED — G
2	Left LED — B
3	Right LED — R
4	Right LED — G
5	Right LED — B
6	Pan servo (SG90)

Ch	Signal
8	Motor R — IN1 (dir)
9	Motor R — IN2 (dir)
10	Motor R — PWM
11	Motor L — IN1 (dir)
12	Motor L — IN2 (dir)
13	Motor L — PWM
14–15	*reserved*

Ch	Signal
7	Tilt servo (SG90)

200 Hz is a compromise between servo timing (ideal 50 Hz) and motor PWM smoothness — it works well for SG90s and the TB6612FNG.

4.4 · Ultrasonic rangefinder

Signal	Pin	Voltage	Function
TRIG	GPIO3 (D2)	3.3 V	10 µs pulse triggers a measurement
ECHO	GPIO4 (D3)	3.3 V	Pulse width encodes distance (RMT RX)
VCC	3.3 V	3.3 V	**3.3 V variant only**
GND	any GND	—	Shared ground

The sensor samples at ~20 Hz. **Obstacle reflex:** if distance < 15 cm, the executor immediately stops and reverses, independent of planner goals.

4.5 · Audio output (MAX98357A I²S amplifier)

The robot speaks through a mono I²S class-D amplifier. Audio is 24 kHz 16-bit PCM — the native output rate of the Gemini TTS model — decoded incrementally into a PSRAM ring so playback starts before the download finishes.

Signal	Pin	Function
BCLK	GPIO7 (D8)	Bit clock
LRCLK	GPIO8 (D9)	Word select / left-right clock
DIN	GPIO9 (D10)	Serial audio data to amplifier
Vin	5 V	Power — see §5 for supply requirements
GND	any GND	Shared ground
SD_MODE	floating	(L+R)/2 — firmware duplicates mono into both slots
GAIN	float	9 dB default

The I²S channel is disabled between utterances — the MAX98357A hisses faintly whenever BCLK is running, so leaving it clocking silence is audible.

> **Trade-off**
> Wiring the amplifier on D8–D10 **replaces the microSD slot**. On the Sense expansion board these three pins are the SD card's SPI bus. There is no alternative pin set — I²S needs a hardware peripheral, so it cannot be moved behind the PCA9685 or the MCP23017.

5 · Power

Power the car from a **2×18650 pack** through an **XL6009 boost converter set to 5 V**. Distribute that 5 V rail to the XIAO 5 V pin, the TB6612FNG (VM + VCC), the PCA9685 (V+ and VCC), the servos, and the **MAX98357A amplifier (Vin)**.

The 3.3 V logic for the OLED, ultrasonic sensor, TCA9548A, and MCP23017 comes from the XIAO's 3V3 pin. Keep

> **Golden rule**
> **Common ground everywhere.** Every module — boost converter, XIAO, motor driver, PCA9685, servos, sensors — must share one GND. Missing grounds cause brown-outs, I²C lockups, and erratic motion.

motor/servo current (high, noisy) on the 5 V rail and logic on 3V3.

Amplifier supply — read this: The MAX98357A draws up to 1 A peaks into a 4 Ω load. With brown-out detection already disabled for motor inrush, an undersized rail will not warn you — it will present as random resets or corrupt audio mid-sentence. Fit a **≥470 µF bulk capacitor** at the amplifier's Vin, plus the usual 0.1 µF close to the pin. Prefer a **separate 5 V feed** from the boost converter to the amplifier rather than daisy-chaining off the motor-driver rail. An **8 Ω speaker** roughly halves peak current versus 4 Ω.

> Brown-out detection is disabled in firmware because motor inrush was tripping it; a stiff 5 V supply and thick power wires matter.

6 · Assembly Steps

1. **Mount the mechanics.** Fit the two gear motors and wheels to the chassis, add the caster/third wheel, and mount the battery holder low and centered.
2. **Wire power first.** Connect the 18650 pack to the XL6009 input, set its output to **5.0 V with a multimeter before connecting anything else**, then run the 5 V and shared GND rails.
3. **Place the XIAO** and bring out I²C (GPIO5/6), STBY (GPIO1), buzzer (GPIO2), ultrasonic pins (GPIO3/4), and I²S (GPIO7/8/9).
4. **Wire the I²C chain:** XIAO SDA/SCL → TCA9548A → PCA9685 (ch0), OLED (ch1), and optionally MCP23017 (ch2). Pull-ups on the breakouts are usually sufficient.
5. **Wire the motor driver:** PCA9685 ch8–13 → TB6612FNG inputs; STBY → GPIO1 motor outputs → the two DC motors; VM/VCC → 5 V.
6. **Add servos** (PCA9685 ch6/7) and **RGB LEDs** (ch0–5, common-anode).
7. **Add the buzzer** on GPIO2 through the 100 Ω resistor, and the ultrasonic sensor on GPIO3/4 (3.3 V power).
8. **Wire the audio path:** MAX98357A BCLK/LRC/DIN → GPIO7/8/9, Vin → 5 V with ≥470 µF bulk cap, speaker → amp output. Leave SD_MODE floating for (L+R)/2.
9. **Double-check the 3.3 V vs 5 V rails** and confirm common ground with a multimeter continuity test before first power-up.

7 · Build & Flash the Firmware

Builds run inside a container — **no local ESP-IDF install is required**. The XIAO uses native USB-Serial-JTAG (USB VID `0x303a`), which `just` auto-detects.

```
# From the repo root
just robocar-unified::build                       # containerized ESP-IDF v5.4 build
PORT=/dev/cu.usbmodem* just robocar-unified::flash # flash over USB-C
just robocar-unified::monitor                      # serial console
# or, in one step:
just robocar-unified::flash-monitor
```

> **Can't enter download mode?**
> Hold **BOOT**, tap **RESET**, then release BOOT to force the bootloader, and re-run the flash command.

The flasher writes three images to an 8 MB, OTA-capable layout:

Offset	Image	Partition
`0x0`	`build/bootloader/bootloader.bin`	bootloader
`0x8000`	`build/partition_table/partition-table.bin`	partition table

Offset	Image	Partition
0x12000	build/robocar-unified.bin	ota_0 (app)

8 · First Boot & Provisioning

8.1 · WiFi over the USB console (Improv Serial)

No WiFi credentials are compiled in. On first boot the device speaks **Improv Serial** on the same USB port you flashed it with – not the BLE variant, so there is nothing to pair. Open the board in a browser-based Improv Serial provisioner (Chrome desktop, e.g. ESP Web Tools) and send your SSID and password. They are written to NVS only after they are proven to connect, so a typo cannot overwrite a working network.

For local development you can instead copy `main/credentials.h.example` to `main/credentials.h` (gitignored) and hard-code credentials.

8.2 · Discovery & AI backend

After connecting, the car is reachable at **`robocar-unified.local`** via mDNS. The planner uses **Gemini Robotics-ER 1.6** to emit goals — `drive()`, `track()`, `rotate()`, and `stop()` — plus `speak()`, which renders a sentence through a second TTS model and plays it while the robot keeps driving.

8.3 · Over-the-air updates

OTA is enabled with app rollback. The updater pulls releases from the `laurigates/mcu-tinkering-lab` GitHub repo; `version.txt` is the single source of truth for the running version and is managed by release-please.

9 · Functional Checkout

Work through these after first flash, watching the serial monitor:

✓	Check	Expected result
☐	Boot log	No PSRAM / boot-loop errors; tasks pin to cores 0 & 1
☐	I²C scan	TCA9548A, PCA9685, and OLED all detected
☐	OLED	Status screen renders (128×64)
☐	LEDs	Both RGB LEDs cycle / show status colors
☐	Buzzer	Audible startup tone
☐	Servos	Pan/tilt center, then sweep within limits
☐	Motors	Both wheels drive forward and reverse; STBY HIGH
☐	Ultrasonic	Distance readings track a hand moving closer/away
☐	Reflex	Car stops/reverses when an obstacle is < 15 cm
☐	Audio / TTS	Robot speaks startup message; speech queues without blocking motion
☐	Microphone	`mic` reports frames arriving, not `gate: DEAF`; `listen` answers a spoken question
☐	WiFi	Provisions via Improv; `robocar-unified.local` resolves

10 · Troubleshooting

Symptom	Likely cause & fix
Boot loop on power-up	Wrong PSRAM mode. The Sense uses **octal** PSRAM (`CONFIG_SPIRAM_MODE_OCT=y`); don't change it.
Random resets under motor load	Weak 5 V rail / missing common ground. Use thicker power wires and verify the XL6009 holds 5 V under load.
No I²C devices found	Not selecting the TCA9548A channel first, or SDA/SCL swapped. Check GPIO5=SDA, GPIO6=SCL.
OLED and PCA9685 conflict	Both bypassing the mux. Route each through its own TCA9548A channel (ch1 / ch0).
Motors don't move	STBY (GPIO1) not HIGH, or VM not on 5 V. Confirm TB6612FNG power and enable line.
Servos jitter	Shared noisy rail. Keep servo power on 5 V with common ground; 200 Hz PWM is expected.
Board won't flash	Force download mode: hold BOOT, tap RESET, release BOOT.
Damaged ECHO / no distance	Used a 5 V HC-SR04. Replace with a 3.3 V module (HC-SR04P).
No audio / distorted speech	Weak MAX98357A supply. Fit ≥470 µF bulk cap at Vin, or use a separate 5 V feed. Check I2S wiring on GPIO7 – 9.

Authoritative pin data: `packages/robocar/unified/main/pin_config.h` · Design rationale: `docs/decisions/ADR-016-hierarchical-ai-controller.md`

Regenerate this PDF with `typst compile --root ../../../.. build-guide.typ`.